

GUÍA 1: DEFINICIÓN PROYECTO APT (FASE 1)

Ecosistema de Software Agéntico KalliCode

Asignatura: Capstone (PTY4614)

Formato aplicado: Arial 12 · Interlineado 1.5 · Texto justificado

Alejandro González Vergara

Ingeniería Informática — Sede San Joaquín

En colaboración con Alloxentric

A. PARTE I

1. Antecedentes Personales

Campo	Información
Nombre estudiante	Alejandro González Vergara
Rut	[Completar RUT]
Carrera	Ingeniería Informática
Sede	Sede San Joaquín

2. Descripción Proyecto APT

Campo	Detalle
Nombre del proyecto	Ecosistema de Software Agéntico KalliCode (en colaboración con Alloxentric) [1.3, 1.4]
Área(s) de desempeño	Desarrollo de Software, Gestión de Infraestructura SRE/DevOps, y Seguridad y Soberanía de Datos [1.3, 1.4].
Competencias	- Ingeniería de Software: Diseño de máquinas de estado observables para gestionar la ejecución de tareas de IA y arquitectura SaaS multi-tenant [1.3, 1.4]. - Seguridad Informática: Diseño de auditoría inmutable mediante firmas criptográficas y hashes encadenados alineados con el reglamento de IA europeo (EU AI Act) [1.3, 1.4]. - Gestión de Datos: Aislamiento multi-tenant estricto mediante Row-Level Security (RLS) en PostgreSQL 16 [1.3, 1.4].

3. Fundamentación Proyecto APT

Relevancia del proyecto APT

La adopción industrial de agentes de Inteligencia Artificial dentro del ciclo de desarrollo de software (SDLC) se enfrenta hoy a una barrera crítica: el control, la trazabilidad y el riesgo inminente de fuga de datos o propiedad intelectual [1.4]. KalliCode aborda directamente esta problemática al diseñar una "Célula de Entrega Agéntica" orientada a entornos corporativos altamente regulados (como los sectores Fintech y Healthtech)

[1.3, 1.4]. En estas industrias, la soberanía de los datos es un requisito legal vinculante y no un aspecto opcional [1.3]. El proyecto es relevante porque redefine el rol del ingeniero informático: se transiciona del tipeo manual de código hacia la gobernanza sistemática de agentes autónomos, protegiendo los activos digitales de la organización mediante una infraestructura determinista, auditable y segura [1.3, 1.4]. El contexto de la problemática se sitúa en una colaboración directa con la empresa Alloxentric, beneficiando tanto a sus células de desarrollo internas como a sus clientes corporativos finales [1.3].

Descripción del Proyecto APT

KalliCode es una plataforma SaaS multi-tenant diseñada para automatizar y coordinar el ciclo de vida del software (SDLC) a través de seis etapas secuenciales: Triage, Design, Build, QA, Security y Deploy [1.4]. A diferencia de los asistentes de código tradicionales, KalliCode utiliza un motor de "CodeMapping" híbrido, que combina grafos de conocimiento en Neo4j (con más de 200k nodos) y capas vectoriales (pgvector/Qdrant) para inyectar un contexto técnico ultrapreciso y libre de alucinaciones a los agentes que operan en la tubería [1.3, 1.4]. Para garantizar la viabilidad económica del pipeline, la arquitectura define un esquema jerárquico de modelos de lenguaje (LLM) de tres niveles: Flash (tareas rápidas de clasificación y baja latencia), Pro (razonamiento lógico intermedio) y Fable (diseño arquitectónico de software complejo y análisis estricto de seguridad), optimizando drásticamente los costos de inferencia operativa [1.3, 1.4].

Pertinencia del proyecto con el perfil de egreso

El proyecto se alinea de manera orgánica con las competencias clave de egreso del Ingeniero Informático de Duoc UC [1.4]. En el pilar de Ingeniería de Software, se valida mediante el diseño de máquinas de estado que estructuran de forma predecible las tareas agénticas [1.4]. En Seguridad Informática, se aterriza mediante el diseño de flujos de auditoría criptográfica ("Three Locks of Corporate Quality") basados en firmas atómicas y encadenamiento de hashes SHA-256 alineados con la EU AI Act [1.3]. En el pilar de Gestión de Datos, la pertinencia radica en el diseño de un aislamiento robusto de base de datos relacional PostgreSQL 16 multi-tenant mediante Row-Level Security (RLS) y un middleware de confinamiento de sesión (`sesion_tenant()`) [1.3, 1.4].

Relación con los intereses profesionales

Como miembro del equipo responsable de las áreas de DevOps, SRE e Infraestructura, mi foco profesional se centra en la eliminación de la fricción operativa y en la construcción de "carreteras de alta fidelidad" (Highways) de CI/CD [1.3, 1.4]. KalliCode me permite llevar a cabo la automatización radical del ciclo de infraestructura, estableciendo un entorno local reproducible y determinista para el equipo de desarrollo [1.3]. En un ecosistema donde los agentes de IA generan código a velocidades sobrehumanas, la infraestructura SRE actúa como la única barrera de control que garantiza que los despliegues progresen de manera segura, observable y sin acumulación de deuda técnica [1.3, 1.4].

Factibilidad de desarrollo del Proyecto APT

El proyecto es plenamente viable durante el semestre académico gracias a la estrategia de "Desbloqueo Estricto" (la infraestructura precede y habilita el desarrollo de la aplicación) [1.3]. La viabilidad se respalda en:

- **Tiempo y Horas:** Planificación estructurada a lo largo de las 18 semanas del semestre [1.4].
- **Materiales:** Uso exclusivo de stacks de código abierto y herramientas contenerizadas (PostgreSQL 16, Redis 7, Docker, Python 3.12) [1.3].
- **Emulación Local sin Costo:** Eliminación de costos de APIs y nubes mediante emulación local: Ollama con Qwen-2.5-Coder para LLM, Azurite para Azure Blob Storage y PostgreSQL local con pgvector [1.4]. Esto elimina dependencias financieras externas y protege la soberanía de los datos durante el desarrollo [1.4].
- **Mitigación de Bloqueos:** Las diferencias de entornos locales del equipo se neutralizan encapsulando la totalidad de los servicios en Docker Compose y Makefiles automatizados, permitiendo un bootstrap completo en menos de 15 minutos [1.3].

B. PARTE II

4. Objetivos

Objetivo General

Establecer la infraestructura de plataforma y los mecanismos de confianza para el Ecosistema KalliCode, garantizando un entorno de desarrollo local automatizado, un aislamiento de datos multi-tenant inexpugnable y pipelines de gobernanza automáticos para habilitar la entrega agéntica segura [1.3].

Objetivos Específicos

- **Aislamiento:** Implementar Row-Level Security (RLS) en PostgreSQL 16 y un middleware de sesión confinada (`sesion_tenant()`) para garantizar un aislamiento absoluto de datos entre tenants [1.3].
- **Automatización:** Configurar la orquestación local utilizando Docker Compose y automatización mediante un Makefile (targets `bootstrap`, `up`, `db`) para lograr un levantamiento completo en menos de 15 minutos [1.3].
- **Gobernanza:** Configurar pipelines de CI/CD automáticos en GitHub Actions con análisis estático estricto (Ruff, estricto Mypy) como puertas de calidad de integración obligatorias [1.3].
- **Inmutabilidad:** Diseñar un esquema de auditoría sellado mediante triggers SQL en la base de datos que bloqueen actualizaciones o eliminaciones (UPDATE/DELETE) en tablas de logs, complementado con hashes SHA-256 encadenados [1.3].

5. Metodología

Descripción de la Metodología y Distribución de Responsabilidades

Se aplica la metodología de "Desbloqueo Estricto" (infraestructura primero), donde la plataforma de confianza debe operar antes de añadir cualquier lógica funcional agéntica [1.3]. El equipo se organiza en fases de trabajo coordinadas [1.3]:

1. **Fase 1: Orquestación (Alejandro González):** Creación del stack local (PostgreSQL 16, Redis 7, Azurite para almacenamiento) y automatización con Makefile [1.3].
2. **Fase 2: Tenencia (Ana Panes):** Lógica relacional multi-tenant, políticas de RLS SQL y pruebas automatizadas de fugas de datos [1.3].
3. **Fase 3: Seguridad (Luis Ibañez):** Triggers SQL inmutables de auditoría, firmas criptográficas para la verificación de los Gates de calidad humana ("Three Locks of Corporate Quality") [1.3].

4. Fase 4: Integración (Grupal): Integración continua, ejecución de suites completas de pruebas unitarias y validación cruzada en CI [1.3].

6. Evidencias de Logro

Tipo de Evidencia	Nombre de la Evidencia	Descripción y Justificación Técnica
Avance (M1)	Makefile y Docker Compose local funcional [1.3]	Archivo de orquestación local parametrizado que levanta PostgreSQL, Redis y Azurite. Garantiza que cualquier miembro del equipo levante el stack idéntico en menos de 15 minutos [1.3].
Avance (M1)	Pipelines de CI con Ruff y Mypy strict [1.3]	Control de calidad automático integrado en GitHub Actions. Previene de forma estricta que la velocidad de generación de los agentes introduzca deuda técnica acumulada [1.3].
Final	Script de pruebas test_no_leak.py [1.3]	Suite de tests unitarios que simulan transacciones cruzadas concurrentes para certificar que el Row-Level Security bloquea el 100% de los intentos de acceso no autorizados [1.3].
Final	Tabla audit_events con triggers inmutables [1.3]	Esquema SQL protegido con triggers que lanzan excepciones inmediatas ante cualquier intento físico de alteración (DELETE/UPDATE) de bitácoras agénticas [1.3].

7. Plan de Trabajo (Hito M1 - 7 Días)

Competencia	Actividad/Tarea	Descripción Actividad/Tarea	Recursos	Duración	Responsable	Observaciones
Ing. Software	Bootstrap del Ecosistema	Crear docker-compose.yml (PostgreSQL, Redis, Azurite) y comandos unificados en Makefile [1.3].	Docker, Python, Make	Días 1-2	Alejandro González	Requiere validar consistencia de puertos de red.
Seguridad	CI Highway Setup	Implementar pipeline de GitHub Actions con análisis de Ruff y Mypy strict como gate de aprobación [1.3].	GitHub Actions, Python	Días 2-3	Alejandro González	Bloquea el merge si existen advertencias o errores de tipado.
Gestión Datos	PostgreSQL Multi-tenant	Modelar esquemas relacionales, habilitar políticas de RLS y middleware sesion_tenant() [1.3].	PostgreSQL 16, FastAPI	Días 3-4	Ana Panes	Es el cimiento crítico para asegurar la soberanía de los datos.
Gestión Datos	Suite de Leak Testing	Desarrollar el script de pruebas automatizado test_no_leak.py [1.3].	pytest, PostgreSQL	Días 4-5	Ana Panes	Valida que ninguna consulta de un tenant intercepte datos ajenos.
Seguridad	Audit Inmutability	Programar los triggers de base de datos que restrinjan operaciones destructivas de auditoría [1.3].	SQL Triggers, DB Admin	Días 5-6	Luis Ibañez	Garantiza que las trazas de logs sean inalterables físicamente.
Seguridad	Chained Hashes	Implementar criptografía SHA-256 para enlazar y sellar decisiones agénticas en los Gates [1.3].	Python Cryptography	Días 6-7	Luis Ibañez	Satisface las directrices de gobernanza del EU AI Act [1.3].
Ing. Software	M1 Clearance	Consolidación y ejecución de la suite de pruebas cruzadas en el repositorio Git unificado [1.3].	Git, GitHub, pytest	Día 7	Todos	Cierre de hito con ejecución de tests exitosa en CI.

8. Carta Gantt Académica (Distribución de las 3 Fases - 18 Semanas)

Actividades / Hitos Clave	Fase 1 (S1 - S4)	Fase 2 (S5 - S12)	Fase 3 (S13 - S18)	Responsables
Hito M1: Bootstrap, contenedores locales y targets en Makefile [1.3].	X (Activo)			Alejandro G.
Hito M1: Confinamiento de base de datos, políticas RLS y test_no_leak.py [1.3].	X (Activo)			Ana P.

Actividades / Hitos Clave	Fase 1 (S1 - S4)	Fase 2 (S5 - S12)	Fase 3 (S13 - S18)	Responsables
Hito M1: Triggers de inmutabilidad en logs y encadenamiento SHA-256 [1.3].	X (Activo)			Luis I.
Fase 2: Integración de grafos híbridos Neo4j, Qdrant e inyección de CodeMapping [1.3, 1.4].		X (Activo)		Todo el equipo
Fase 2: Implementación y orquestación de agentes con tiers de LLM (Flash, Pro, Fable) [1.3, 1.4].		X (Activo)		Todo el equipo
Fase 3: Pruebas de estrés concurrentes, QA agéntico, despliegue SaaS y Defensa de Título [1.3, 1.4].			X (Activo)	Todo el equipo

9. Project Reflection (English Only)

Transcripción textual del entregable obligatorio en inglés:

The development of KalliCode represents a major architectural paradigm shift: moving from manual code typing to systematic agentic code generation and pipeline orchestration [1.4]. In an agentic software factory, the engineer's core value is no longer in writing routine code blocks, but in defining precise constraints, establishing high-fidelity operational contexts (CodeMapping), and building rigorous validation highways [1.4]. AI agents possess immense generative velocity, yet speed without governing systems creates technical debt 'at the speed of light' [1.3]. Thus, implementing the 'Three Locks of Corporate Quality' (Human Governance Gates) paired with cryptographic immutable logs is not an optional security feature; it is the absolute foundation that permits a modern corporation to trust an autonomous pipeline with its digital assets [1.3].

Traducción al español (solo para tu comprensión):

El desarrollo de KalliCode representa un cambio de paradigma arquitectónico fundamental: pasar del tipeo de código manual a la generación sistemática de código agéntico y a la orquestación de pipelines [1.4]. En una factoría de software agéntico, el valor central del ingeniero ya no reside en escribir bloques de código rutinarios, sino en definir restricciones precisas, establecer contextos operativos de alta fidelidad (CodeMapping) y construir carreteras rigurosas de validación [1.4]. Los agentes de IA poseen una velocidad generativa inmensa, pero la velocidad sin sistemas de gobernanza crea deuda técnica "a la velocidad de la luz" [1.3]. Por lo tanto, implementar los "Three Locks of Corporate Quality" (puertas de gobernanza humana) junto con registros criptográficos inmutables no es una característica de seguridad opcional; es la base absoluta que permite a una corporación moderna confiar sus activos digitales a una tubería autónoma [1.3].

10. Individual Conclusions (English Only)

Transcripción textual del entregable obligatorio en inglés:

Automation functions as the ultimate unlocker in modern platform engineering [1.3]. By delivering a 100% local, zero-cost development stack reproducible in under 15 minutes, we have eliminated environment discrepancy and initial setup friction [1.3, 1.4]. The platform acts as a secure, observable playground for agentic execution [1.3]. This experience demonstrates that successful corporate AI adoption is not about selecting a

powerful model; it relies on building a robust surrounding ecosystem that can protect data sovereignty (RLS), enforce static quality gates (Ruff/Mypy), and maintain immutable records [1.3, 1.4]. This infrastructural governance is the future of DevOps/SRE, moving the engineer from manual coders to high-level system architects [1.3, 1.4].

Traducción al español (solo para tu comprensión):

La automatización funciona como el desbloqueador definitivo en la ingeniería de plataformas moderna [1.3]. Al entregar un stack de desarrollo 100% local, de costo cero y reproducible en menos de 15 minutos, hemos eliminado la discrepancia de entornos y la fricción del setup inicial [1.3, 1.4]. La plataforma actúa como un área de pruebas segura y observable para la ejecución agéntica [1.3]. Esta experiencia demuestra que la adopción corporativa exitosa de la IA no se trata de seleccionar un modelo de lenguaje potente; depende de la construcción de un ecosistema circundante robusto capaz de proteger la soberanía de los datos (RLS), aplicar puertas de calidad estáticas (Ruff/Mypy) y mantener registros inmutables [1.3, 1.4]. Esta gobernanza de infraestructura es el futuro de DevOps/SRE, moviendo al ingeniero de codificadores manuales a arquitectos de sistemas de alto nivel [1.3, 1.4].

Verificación de Calidad del Documento Word Generado

El archivo de Word descargable que tienes disponible ha sido construido mediante un script automatizado que aplica directamente sobre el archivo binario las propiedades solicitadas:

- **Fuente:** Seteada globalmente en Arial con tamaño 12 para cuerpo y jerarquías formateadas para títulos.
- **Alineación:** Párrafos de texto justificados (WD_ALIGN_PARAGRAPH.JUSTIFY) para mantener la simetría visual técnica.
- **Interlineado:** Configurado estrictamente en 1.5 líneas en las propiedades de formato de párrafo.
- **Tablas:** Con anchos de columna definidos celda por celda para evitar deformaciones en Google Docs o Microsoft Word, incluyendo colores de contraste institucionales para encabezados.